ChatGPT

Semantic Graph-Powered Podcast Recommender

Authors: Dinis Cruz and ChatGPT Deep Research

Introduction

Podcasts contain a wealth of knowledge and insights, but much of this content remains difficult to navigate and utilize after listening. Listeners often finish a long podcast remembering only fragments, with the connections between ideas easily lost [1]. Traditional recommendation methods (e.g. collaborative filtering or keyword search) struggle to capture the rich semantic relationships within podcast content. To address these challenges, we propose a *semantic knowledge graph*-powered podcast recommender system. This approach uses **knowledge graphs** to represent podcast content and user preferences, enabling highly personalized and **explainable** recommendations. By leveraging recent advances in large language models (LLMs) and graph databases, our system can automatically generate a structured semantic graph for podcast episodes and use it to recommend relevant content with full transparency. This white paper outlines the motivation, architecture, and technical implementation of the system, which is inspired by the MyFeeds.ai platform's serverless graph architecture.

Why Knowledge Graphs? In AI systems that assist with complex tasks – from enterprise Q&A to personalized feeds – properties like provenance, determinism, and explainability are not luxuries but requirements [2]. Conventional vector-based semantic search approaches (using embeddings in a vector database) can capture broad topical similarity, but they introduce an *"interpretability bottleneck"* that hinders traceability [3]. In contrast, a graph-based strategy stores knowledge as explicit nodes and relationships, a format inherently aligned with human reasoning and oversight [4]. By representing podcast information as a knowledge graph, we can trace exactly why a recommendation was made (e.g. shared topics or entities between episodes) and ensure each connection is grounded in the source content. This graph-centric approach also avoids the approximation issues of vector similarity search – recommendations are based on exact semantic relationships rather than opaque high-dimensional math. Importantly, recent work has shown that graph-based personalization can be highly effective at scale. For example, Spotify reported double-digit improvements in audiobook engagement after deploying a graph-powered recommendation model for audiobooks and podcasts [5]. These successes reinforce that a knowledge graph approach can meet the strict requirements for trustworthy, **AI-driven recommendations**.

In the following sections, we provide a deep technical dive into the system's design. We first outline the overall use case and user experience. Next, we describe the end-to-end workflow for ingesting podcast data and constructing semantic knowledge graphs, including how an LLM is used to bootstrap the graph creation. We then explain how personalized user **persona graphs** are built and leveraged to match users with relevant podcast episodes or even specific segments. The paper also details the serverless architecture underpinning the system – in particular, a caching and storage mechanism that ensures every intermediate result is saved with full provenance, enabling reproducibility and iterative refinement of the knowledge graph. Finally, we discuss how this architecture enables continuous improvement through feedback loops and future extensions such as full audio transcript analysis.

Use Case and Vision

The primary use case for this system is a **podcast recommendation engine** that can suggest podcast episodes (or snippets of episodes) tailored to a user's interests. Unlike a typical podcast app which might recommend shows based on broad genres or "listeners also subscribed to X," our system can make fine-grained recommendations based on the actual *content* of the episodes. For example, consider a user who is particularly interested in the topic of semantic knowledge graphs in AI. Using our system, this user could receive a list of recommended podcast episodes (across various shows) that all delve into knowledge graphs, even if those episodes are buried in hour-long podcasts with diverse topics. Moreover, the system could highlight **why** each episode is recommended – e.g. *"Episode A discusses knowledge graphs and vector databases, which aligns with your interest in explainable AI"*. This level of transparency is made possible by the underlying graph representation connecting topics and entities.

Another compelling aspect is delivering **segment-level recommendations**. Rather than always suggesting entire episodes, the system can identify and extract the most relevant segments of an episode for a given user. For instance, if a 60-minute podcast only has a 5-minute segment about a topic the user cares about, the system could surface just that segment as a recommendation. This is especially useful for users who follow specific subjects or people; they can directly jump to the parts of podcasts that interest them most. In future iterations, one could imagine automatically generating summary snippets or even slideshows ("executive summaries") of those segments. The ultimate vision is that users have a personalized, interactive feed of podcast knowledge – curated from raw audio into a navigable graph of insights, topics, people, and stories. Users could query this personal knowledge base with questions or simply explore connections (e.g. "show me how concept A from one podcast relates to concept B from another"), turning passive listening into an active learning experience.

From the content creator's perspective, this approach highlights the value of rich metadata. Today, most podcast RSS feeds include only basic information (title, description, maybe tags). Our system compensates by generating semantic metadata automatically. However, in the future, podcast creators themselves could provide high-quality semantic annotations or knowledge graphs for their episodes. This would ensure even more accurate recommendations and unlock new content experiences (for example, dynamic podcasts that reconfigure based on the listener's profile). Until that happens, our system serves as a bridge – using AI to produce the missing semantic structure from unstructured audio and text.

System Overview and Architecture

At a high level, our podcast recommender system operates in two phases: an **offline ingestion phase** where podcast content is fetched and transformed into a knowledge graph, and an **online recommendation phase** where user queries (or feed requests) are answered by traversing the graph. The entire system is built in a **serverless, API-driven fashion** – all functionality is exposed via RESTful endpoints (built with FastAPI), and all data is stored as files in cloud object storage rather than in a traditional database. This design maximizes scalability and transparency, as every piece of data (from raw RSS feeds to the final graphs) is versioned and persisted for inspection. Figure 1 below illustrates the major components and data flow:

```
          .-------------------------------------------------.
          |          **Semantic Podcast Recommender**       |
          |                                                 |
```

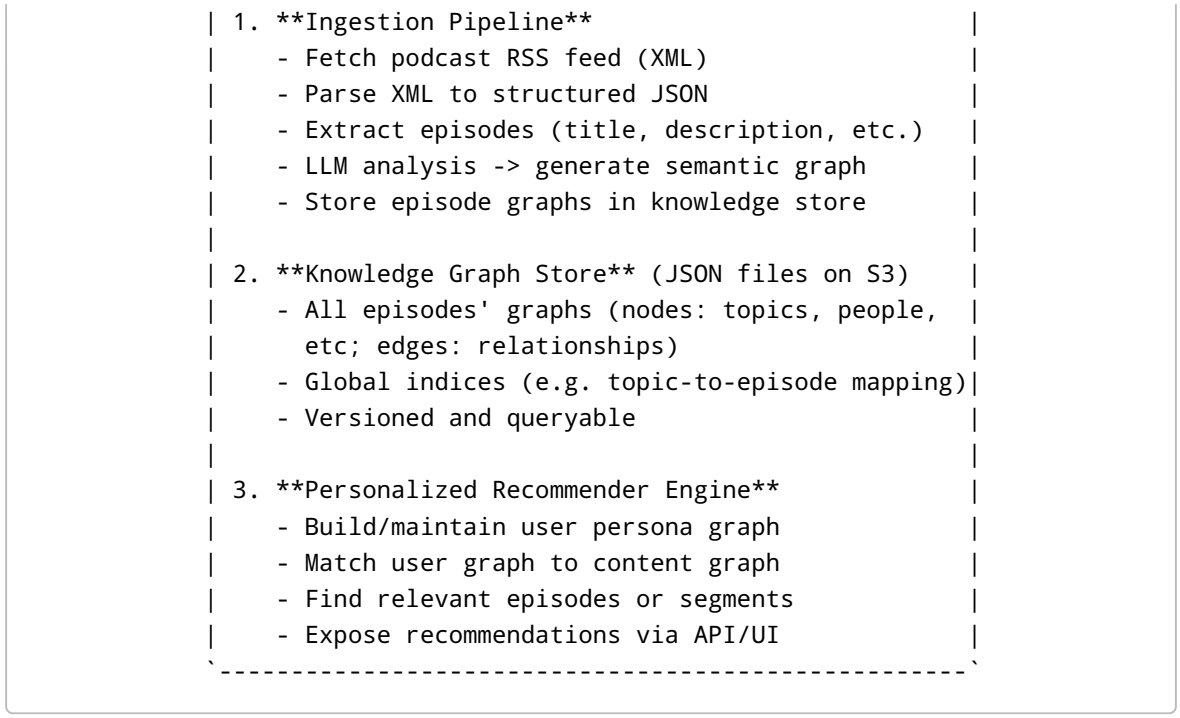

Figure 1: *High-level architecture of the semantic graph-powered podcast recommender.* The ingestion pipeline (left) processes podcast feeds into structured knowledge graphs. The knowledge graph store (center) accumulates these graphs with full versioning and provenance. The recommender engine (right) matches users to content via graph traversal, providing explainable recommendations.

In phase 1 (ingestion), the system continuously or periodically fetches new podcast data and enriches it. This could be triggered by events (e.g. a new episode published) or on a schedule. Each podcast's RSS feed (an XML document) is retrieved and cached. The feed is then parsed to extract episode metadata. Initially, we focus on the **textual metadata** (episode titles and descriptions) as the content source. An LLM is employed to analyze each episode's description and generate a semantic representation – essentially a mini knowledge graph capturing the key topics, entities (people, places, companies), and their relationships mentioned in the description. This results in a **per-episode knowledge graph** (stored as a JSON file, for example). As new episodes are processed, their graphs are added to a growing collection. The system can then derive higher-level graphs from these, such as a *topic graph* that connects episodes via shared topics, or a *show-level graph* summarizing an entire podcast series. Each of these artifacts is saved back to the **knowledge graph store** in cloud storage.

In phase 2 (recommendation), a user's interests are represented as a **persona graph**. This persona graph could be built explicitly (e.g. the user selects topics of interest, or connects their profile which we convert to a graph), and/or implicitly learned from their listening history. The recommendation engine takes this persona graph and queries the content knowledge graph to find the best matches. Because both user preferences and content are in the same semantic graph space, matching can be done via straightforward graph traversal and pattern matching (no neural network black boxes needed at query time). For example, if the persona graph has a node for *"semantic web"* connected to *"interest: high"*, the engine might look for episodes whose graphs have a *"semantic web"* topic node, and rank those highly. It can also traverse two hops: if the user likes concept A which is related to concept B in the knowledge graph, it might recommend episodes about B as well (analogous to "if you liked X, you may also like Y"). This flexibility in traversal allows discovery of content that is not an obvious keyword match but is conceptually related. Crucially, every recommendation can be **explained** by tracing the path in the graph – e.g. *"Recommended because Episode 5 shares the topic 'Knowledge Graphs' and the person 'Tim*

Berners-Lee' with something you liked". The result is a personalized feed of podcast content that the user can trust and refine.

Content Ingestion and Knowledge Graph Construction

Turning raw podcast data into a structured knowledge graph is the core of our system's pipeline. This process is composed of a sequence of transformations, each of which is implemented as an **idempotent, cache-enabled workflow**. The design principle is that every step takes some input (from a previous step or an external source), performs a **pure transformation**, and writes its output to the persistent store. By caching each intermediate result, the system ensures that work is not repeated unnecessarily and that every piece of data can be inspected or recovered for debugging. Below, we break down the main stages of the ingestion pipeline:

1. **RSS Feed Retrieval and Caching:** The pipeline begins by fetching the podcast's RSS feed (an XML file listing recent episodes and metadata). This is done via a simple HTTP GET request to the feed URL. We utilize a caching service that immediately **saves the raw RSS XML** to the cloud storage database. The file is stored under a structured path (for example, `feeds/{podcast_name}/rss/latest.xml`), and also under a timestamped path (e.g. `feeds/{podcast_name}/rss/2026-01-20T14-20-00.xml`). Storing a timestamped copy allows versioning: if the feed changes (new episodes added), we will retain the old versions as well. The caching layer can detect if the newly fetched feed is identical to the last saved version (using a checksum or file diff). If there is no change, the pipeline can short-circuit further processing for that feed (no need to reprocess all episodes every time). If there *is* a change (e.g. a new episode appears), we proceed to the next steps for just the new content.

2. **Parsing XML to JSON:** The raw RSS XML is parsed into a structured JSON (or Python dictionary) representation. This transformation essentially converts the XML tree (with items for each episode) into a more convenient form for downstream processing. We then cache this parsed output (e.g. `feeds/{podcast_name}/parsed/latest.json`). At this stage, the data is still quite general – it contains all the fields present in the RSS (title, description, publication date, maybe duration, etc.). The next step will distill only the information we need for building our graphs.

3. **Episode Metadata Extraction:** From the parsed feed, we extract the list of episodes along with key metadata fields of interest. Primarily, we take the episode **title** and **description** (and possibly the publication date or unique ID). These are packaged into a simpler JSON structure, for example:

```
{
  "podcast": "Sample Podcast Show",
  "episodes": [
    {
      "id": "episode123",
      "title": "Knowledge Graphs in AI",
      "description": "In this episode, we discuss how knowledge graphs can
enhance AI systems ...",
      "pub_date": "2025-12-01"
    },
    {
```

```
      "id": "episode124",
      "title": "Vector Databases vs Graph Databases",
      "description": "This talk explores the differences between vector
search and graph-based approaches ...",
      "pub_date": "2025-12-15"
    }
    // ...more episodes
  ]
}
```

This extraction makes it easy to iterate over episodes for content analysis. We save this structured episode list to the cache as well (e.g. `feeds/{podcast_name}/episodes/latest.json`). By saving it, if an upstream step changes (say the feed was updated), we can quickly diff which episodes are new or modified and only process those going forward.

1. **LLM-Powered Semantic Graph Generation:** This is the most critical step – converting unstructured text (the episode description, and later the transcript) into a structured knowledge graph. For each episode, the system invokes a Large Language Model to analyze the description and identify the key entities and relationships. We prompt the LLM with a carefully crafted instruction, such as: *"Extract the key topics, entities (people, organizations, etc.), and any notable relationships or facts described in this podcast summary. Represent these as a set of triples or a JSON graph structure."* The LLM (e.g. GPT-4 or similar) will output a structured representation. For example, from an episode description about knowledge graphs in AI, the LLM might produce something like:

```
{
  "entities": [
    {"id": "Knowledge Graphs", "type": "Topic"},
    {"id": "AI Systems", "type": "Topic"},
    {"id": "Vector Databases", "type": "Technology"},
    {"id": "Graph Databases", "type": "Technology"},
    {"id": "Jane Doe", "type": "Person"}
  ],
  "relations": [
    {"source": "Knowledge Graphs", "target": "AI Systems", "relationship":
"enhance"},
    {"source": "Graph Databases", "target": "Vector Databases",
"relationship": "compared_to"},
    {"source": "Jane Doe", "target": "Knowledge Graphs", "relationship":
"discusses"}
  ]
}
```

In a graph database terminology, the above could be nodes and edges. Here we are storing it as JSON for simplicity and later conversion to a graph structure. We call this output the **episode semantic graph**. It captures facts like *"Jane Doe discusses Knowledge Graphs"* and *"Knowledge Graphs enhance AI Systems"* as first-class data. Note that the LLM is effectively doing a summarization and entity-relation extraction task. It may not be 100% accurate – it could miss some entities or introduce a hallucinated relationship. However, a key principle of our system is that **we don't require the initial graph to be perfect**. We prefer to get a decent first draft and then improve it over time (with validation and

feedback loops, described later). All LLM outputs are stored (e.g. `graphs/{podcast_name}/{episode_id}_graph.json`) to provide a paper trail of how each graph was derived.

The use of LLMs here essentially automates what would otherwise be a manual metadata creation process. Previous attempts at building podcast knowledge bases would falter due to the lack of structured data – transcripts are messy and descriptions are brief. By applying an LLM, we leverage its understanding of language to infer structure. It's important to mitigate typical LLM pitfalls in this step. For instance, LLMs can hallucinate facts or incorrectly link entities [6]. Our system tackles this by keeping the human in the loop: we maintain provenance for every triple (e.g. we could attach the sentence or phrase from the description that led to that triple). If something looks suspect or if an entity is unknown, we can later validate it or have a user confirm it. This approach aligns with emerging best practices for trustworthy LLM-generated knowledge, which emphasize *architectural constraints* (using defined vocabularies, validation rules, and keeping track of sources) to curb hallucinations [7] [8]. In our implementation, we start with a base ontology of common entity types (topics, persons, organizations, etc.) and relationship types. The LLM is instructed to use this schema when outputting the graph. This "ontology of ontologies" approach means we can evolve a global schema over time as new types of information appear, but maintain consistency across all episode graphs.

1. **Graph Storage and Indexing:** Once the episode's semantic graph is generated, it is saved to the **graph store**. The graph store in our system is simply a structured set of JSON files in cloud storage (e.g. Amazon S3), but it conceptually acts as a **graph database**. For instance, we might have a directory `graphs/{podcast_name}/episodes/` containing all episode graph files. We also maintain some **global graph indexes** to aid querying. One such index is a *topic-to-episodes mapping* – effectively a graph where nodes are topics and they have edges to episodes in which they appear. We build this by scanning each episode graph for entities of type "Topic" (or any chosen key type) and updating a central index (this could be a JSON file or a lightweight database table) that maps each topic to the list of episodes (and even the specific relationship context). Another index could map *people* to episodes, etc. These indices act like materialized views of the overall knowledge graph and allow quick lookup (e.g. quickly find all episodes about **machine learning**). All these are derived and saved as part of the pipeline, meaning they are updated incrementally whenever new episodes are processed. By doing this work at write-time (when new data arrives), we optimize the read-time performance – when answering user queries, we don't need to recompute these relationships on the fly.

2. **Iterative Refinement (Feedback Loops):** The pipeline described above produces a first version of the knowledge graph. However, it's designed with continuous improvement in mind. Because every intermediate and final output is stored with provenance, we can introduce review steps to refine the data. For example, an automated process or a human curator could periodically scan the graphs for obvious errors (like a relationship that doesn't make sense) and correct them. Users of the system can also provide feedback: if a user says a particular recommendation was irrelevant, that signals that some connection in the graph might be too weak or incorrect, which we can then adjust (e.g. downgrading the weight of that connection or marking it as invalid). The system's architecture supports this in a couple of ways. First, since the data is all in a graph format, corrections can be as simple as adding or removing an edge, or merging two nodes that were mistakenly separate (say "USA" and "United States" if the LLM didn't recognize them as the same). Second, any change can be logged as a new version of the graph, preserving the old version. This gives us a full history of how the knowledge graph evolves as the system learns. Over time, one would expect the graphs to converge to higher accuracy and richness, especially as more episodes get processed and common patterns emerge. The presence of **shared nodes** (like recurring topics or persons across episodes) means the graph gains a kind of memory – e.g. once "Graph Databases" is established as an entity node in one context, another episode that

mentions “graph database technology” could be linked to that same node, rather than creating a duplicate. This is how we achieve a *network effect* in the data: each new piece of content reinforces and expands the overall knowledge network, rather than living in isolation.

3. **(Optional) Full Audio Transcription Processing:** Thus far, we have focused on using the episode descriptions as the content source. The next step – which we consider a Phase 2 extension – is to incorporate the **podcast audio itself**. This involves downloading the audio file of the episode, running it through a speech-to-text engine to obtain a transcript, and then applying the same LLM-based analysis on the transcript (or sections of it) to extract more granular knowledge. Processing full transcripts presents challenges (they are long, often 30-60 minutes of speech, which is thousands of words), but it offers a much deeper understanding of the content. Our architecture is well-suited for this extension: we would treat the transcript as another input to the pipeline (likely chunking the transcript into segments and processing each segment). The output could be a **segment-level knowledge graph**. For example, a 60-minute episode might be broken into, say, 6 segments of 10 minutes, each summarized and graphed. The episode’s overall knowledge graph could then reference these segments. For the user, this unlocks the ability to navigate inside episodes. The recommendation engine could suggest not just “Episode X” but “Episode X – Segment 2 (10:00–20:00): discussion on Knowledge Graph vs Vector DB”. The segment’s graph would tell us it covers those topics, and we could even auto-generate a summary or a set of bullet points for that segment. While transcript analysis is computationally more intensive (and potentially expensive when using LLMs), it can be done asynchronously and incrementally. Additionally, as LLMs improve and specialized models for summarization become available, this process will become more efficient. Overall, integrating transcripts would significantly enrich the knowledge graph, making connections that might not be evident from the brief descriptions alone. It aligns perfectly with our goal of turning every piece of podcast knowledge into a node or edge in the graph of ideas.

Personalized Recommendation Engine

With the content knowledge graph in place, we shift to the user side of the equation: constructing a **persona graph** for each user and utilizing it to deliver recommendations. The persona graph is a semantic representation of the user’s interests, preferences, and context. It can be built in multiple ways. One approach is explicit curation: during onboarding, a user might select topics they are interested in (e.g. *“Artificial Intelligence”*, *“Cybersecurity”*, *“Startups”*). These become nodes in their persona graph, possibly with a high weight or a tag like “interest: yes”. Another approach is to infer the persona graph from behavior: if a user listens to a lot of episodes about *cryptography*, the system can add that as a node in their graph (or increase its weight). The persona graph could also include negative preferences (topics to avoid) or contextual information (e.g. the user’s profession, which might indicate domain-specific interests). The key is that the persona graph uses the **same vocabulary** as the content graph – it has nodes that correspond to topics, people, or other entities that also appear in episode graphs. This alignment is what allows matching: we essentially look for overlaps or connectable paths between the user graph and the content graph.

Graph Matching for Recommendations: The recommendation engine performs a graph query to find candidate episodes for the user. In simple terms, we look for episodes whose graph has strong connections to the user’s persona graph. This could be a direct overlap (e.g. a topic node that is in both the user graph and an episode graph) or a short two-hop connection (e.g. the user likes *X*, episode has *Y*, and X and Y are related in the global knowledge graph). Because we built indices (like topic-to-episode maps), the engine can quickly fetch all episodes related to a given interest. It then can rank or filter them. Ranking criteria might include: how many matching nodes between user and episode, the importance of those nodes (for instance, a match on a key interest like “Machine Learning” might count

higher than a match on a minor entity), recency of the episode (some users prefer newer content), and even popularity if desired. We can also incorporate the user's listening history to avoid repeats or to boost things similar to what they liked previously. Since all this data is structured, these algorithms are transparent and tunable; we're not dealing with an inscrutable machine-learned matrix, but explicit features.

One powerful capability of graph-based recommendations is the ability to **explain and justify** results. As mentioned earlier, any recommended item comes with a subgraph as an explanation. The system can present a visualization or a text explanation like: *"Recommended because you showed interest in Quantum Computing, and this episode covers Quantum Computing and Cryptography (which is related)."* If the user has a persona node for a favorite speaker or podcast host, the system could say *"Recommended because it features an interview with Alice Smith, whom you follow."* This not only builds trust but also allows the user to give precise feedback – for example, *"I'm not actually interested in Cryptography, please de-prioritize that"*. The persona graph would then update (perhaps lowering the weight of that node or removing it), and future recommendations would adjust accordingly. This kind of fine-grained feedback loop is rarely possible in traditional recommenders. In our system, it's a natural consequence of the graph architecture: the user can edit their graph (directly or indirectly) to reflect their true preferences, and the system will respond with new graph query results.

User Interface Considerations: While the core of our system is API-first (the recommendation results can be retrieved as JSON via an API), the UI can be quite innovative given the data we have. We can create visualizations of the user's persona graph and the content graphs. For example, a user could see a visual knowledge map of their interests and how recommended episodes connect to them. Early prototypes in MyFeeds demonstrated **per-topic and per-person dashboards** – essentially custom mini-sites for a given topic or individual that aggregate relevant content. It's feasible to dynamically generate a page like "MyFeed for Topic X" which shows top episodes, latest news, related topics, etc., all driven by the underlying graph data. Because the system is built on standard web technologies (FastAPI for the backend, and it could use any web frontend), these UIs can be generated quickly and even tailored to each user. The important point is that, regardless of UI, the heavy lifting is done by querying the graph store via the API, meaning the logic is centralized and consistent. A mobile app, a web dashboard, or even a chatbot interface could all tap into the same personalized recommendation endpoint.

The persona graph approach also opens the door to **multi-user or community graphs**. While not the focus of this paper, it's worth noting that if we have multiple users each with their own graph, we could identify commonalities and trends. For example, a "group graph" could be unioned from several individuals to recommend a podcast interesting to all members of a team. Or a user could choose to explore another persona graph (say, an expert's profile) to find content that person would likely recommend. This kind of *graph-based collaborative filtering* is an intriguing possibility, achieved not by matrix factorization but by literally overlapping graphs and seeing intersections.

Serverless Architecture and Data Provenance

A distinguishing feature of our implementation is the **serverless, cache-centric architecture**. Instead of using traditional databases or stateful servers, we treat cloud storage (like AWS S3 or Azure Blob Storage) as the backbone of our data layer. All components – from data ingestion to the recommender API – are stateless or lightly stateful services that read from and write to this storage. This design brings several benefits: scalability (storage is virtually unlimited and auto-scaling), cost-efficiency (pay-as-you-go for storage and compute, with no always-on database instance), and crucially, **provenance and reproducibility**. Every piece of data that flows through the system is stored with a unique key, making it

possible to trace how a particular output (say, a recommendation given on January 20, 2026) was derived.

The core of this architecture is the **caching service** (also referred to as the *cache database*). This is a custom service layer built on an open-source library called MemoryFS (Memory Filesystem) which abstracts away the details of where files are stored. In development, one can configure MemoryFS to use the local disk or even in-memory storage; in production, it can point to an S3 bucket. The code that uses the caching service simply calls functions like `cache.save(path, data)` and `cache.load(path)` without worrying about the underlying medium. The cache enforces a directory convention for organization. For example, data might be organized as:

```
myfeeds-data/
   feeds/
      {podcast_name}/
         rss/
            latest.xml
            2026-01-20T14-20-00.xml
         parsed/
            latest.json
         episodes/
            latest.json
   graphs/
      {podcast_name}/
         episodes/
            {episode_id}_v1.json
         consolidated/
            latest.json
   index/
      topics_to_episodes.json
```

In the above structure, you can see how each step of processing has a place in the hierarchy. The caching system can also store *metadata* alongside each file (for instance, a hash of its content, a timestamp, etc.) which is used to detect changes. When a new RSS feed is fetched, we compare its hash to `rss/latest.xml` to decide if we need to process further. If a new episode graph is generated, we might increment a version number in its filename (as shown with `_v1.json`), allowing multiple versions if we re-generate it with improved prompts or data.

Our **flat workflow** design pattern ties in closely with this cache. Each workflow (for example, “Parse RSS” or “Generate Graph for Episode”) is implemented as a function or API endpoint that does the following: (a) Load the necessary input from cache, (b) perform its transformation, (c) save the result to cache, and (d) return the path or identifier of the saved result. Because the result is saved, any other part of the system can now use it without recomputation. It also means if the process crashes or is interrupted, it can resume later from where it left off by checking which outputs already exist. This resiliency is important for long pipelines like processing dozens of episodes – we don’t want to restart from scratch if something fails in the middle. It’s akin to the concept of *checkpointing* in data engineering.

The **LLM service** is another component in our architecture. Rather than calling an external LLM API (like OpenAI) directly from the pipeline code, we have an intermediary service that handles all LLM requests. This service encapsulates details like API keys, model selection, rate limiting, and error handling. When

the ingestion pipeline needs a description analyzed, it sends a request to the LLM service (for example, a POST request with the text). The LLM service then returns the structured graph output. This separation has a few advantages: it decouples the pipeline from any specific LLM vendor or model (we could switch from one LLM API to another, or even use a local model, by modifying the LLM service without changing the pipeline code). It also enables caching of LLM outputs – the LLM service can implement its own cache such that identical requests don't incur repeated costs. In our system, we indeed cache LLM responses keyed by a hash of the input prompt. Given that podcast descriptions sometimes repeat boilerplate text or we might re-run the pipeline with the same data, this can save time and money.

Deployment-wise, each service (the main pipeline service, the LLM service, and the recommendation API service) can be deployed as **serverless functions** or as lightweight containers in the cloud. For instance, using AWS, we might deploy these as Lambda functions behind API Gateway endpoints, or as Docker containers on a Fargate cluster, depending on performance needs. The important point is that none of these services maintain long-term state – all state lives in the storage. This aligns with the serverless philosophy and makes horizontal scaling trivial (you can run multiple instances of the recommendation API, and they'll all read the same cached data from S3). We have a continuous integration (CI) pipeline set up to automate deployment: whenever we update the code (which is kept in a version control repository), a new build is triggered, tests run, and the service is deployed to the cloud environment. This automation ensures that improvements or bug fixes to the pipeline or models can be rolled out quickly.

Provenance and Determinism: Because every answer our system gives can be traced back to stored data, we achieve a level of determinism and auditability that is crucial for trust. If a user asks "Why did you recommend this podcast to me?", we can pull up the exact subgraph that led to it. If an administrator wants to verify the system's output, they can reproduce a result by following the chain of cached files (starting from the RSS input, through parsed JSON, through the LLM output, to the final recommendation). This is invaluable not only for debugging and improving the system, but also for potential compliance or moderation. For example, if a podcast producer challenges why their episode was or wasn't recommended for certain topics, we have the data to show the reasoning. This addresses a common problem with AI systems: the "black box" effect. By design, our architecture avoids black boxes – even the LLM's contributions are treated as data that can be reviewed and, if necessary, corrected or overridden. As one expert noted about LLM-built knowledge graphs, issues like hallucinations and missing provenance are *predictable failures* when systems lack the proper scaffolding [6]. Our approach directly tackles that by weaving provenance into the fabric of the system. Each node and edge in the graph could carry metadata about its source (e.g., which podcast and even which sentence it came from). This way, the knowledge graph is not just a product of AI, but a verified, evolving dataset that can be trusted for downstream use.

Conclusion

In this white paper, we presented a comprehensive design for a semantic graph-powered podcast recommender system. By transforming unstructured podcast content into a structured knowledge graph, the system enables **deep personalization** and **transparent recommendations** that go far beyond what traditional methods can offer. We described how a combination of serverless architecture, caching of every data transformation, and LLM-assisted graph construction makes it feasible to build and maintain such a knowledge-rich system. The approach addresses key challenges in modern AI-driven applications: ensuring provenance and determinism (every recommendation is traceable to sources), achieving explainability (recommendations can be explained via graph relationships), and allowing adaptability (the graphs can be refined over time with feedback).

Our implementation leverages open-source technologies and standards, aligning with broader trends in the industry. The use of knowledge graphs for AI is gaining momentum as organizations recognize the limitations of purely vector-based or end-to-end learned systems [3]. Instead, a hybrid approach is emerging: LLMs are used to assist in constructing structured knowledge (where they excel at language understanding), and graph databases or graph structures are used to store and reason with that knowledge (where they excel at logical consistency and queryability). This system exemplifies that pattern. It also demonstrates that one does not need a massive infrastructure to get started – the serverless design means even a small team or community could implement this by gluing together cloud services and open-source libraries. In fact, all data in our system is essentially stored as JSON files in cloud storage, which lowers the barrier to entry and maintenance.

The potential applications of this architecture extend beyond podcast recommendations. It could be applied to any domain where unstructured content (text, audio, video) contains latent knowledge that users want to surface in a personalized way. For example, it could power a research assistant that builds a knowledge graph from academic paper abstracts and suggests relevant papers to scientists. Or it could ingest news articles and help readers explore stories by understanding the entities and narratives across sources. The key innovation is treating content ingestion as a graph-building exercise and treating recommendations as a graph query problem. This provides a high level of **control and visibility** – qualities that are increasingly important as AI systems are deployed in sensitive and trust-dependent contexts.

In conclusion, the semantic graph-powered approach offers a path forward for **next-generation recommendation systems**: ones that users can trust and engage with, content creators can contribute to, and developers can extend. We invite the community to explore these ideas, implement and adapt them, and collaborate on refining the ontologies and tools that make such systems possible. By sharing this design (credit to *Dinis Cruz and ChatGPT Deep Research* for its authorship), we hope to accelerate the development of rich, open knowledge graph solutions that make information from podcasts (and beyond) more accessible and useful to everyone.

Sources:

1. [1] *RW #8 - Turning Podcasts Into Knowledge Graphs* – Discussion on how podcast knowledge is often ephemeral and the motivation to capture relationships in a knowledge graph (David Andrés, *MLpills* substack, 2025).
2. [2] *Why Vector Databases Fail in LLM Projects: A Case for Graph-Based Retrieval* – Notes that conventional vector-based semantic search introduces an "interpretability bottleneck," whereas graph-based strategies align with human reasoning by using explicit nodes and relationships (Dinis Cruz, LinkedIn post, 2023).
3. [5] *Personalizing Audiobooks and Podcasts with Graph-Based Models* – Case study from Spotify showing a 23% increase in audiobook streams and 46% increase in new audiobook starts after deploying a graph-powered recommendation model, illustrating the efficacy of graph-based approaches (Spotify Research, 2024).
4. [6] [7] *Can LLMs Build Trustworthy Knowledge Graphs? Yes, With the Right Architecture* – Explanation of problems like hallucinations and missing provenance in unconstrained LLM-generated knowledge graphs, emphasizing the need for architectural constraints and provenance tracking (Louie Franco III, LinkedIn article, 2025).

---

[1] RW #8 - Turning Podcasts Into Knowledge Graphs
https://mlpills.substack.com/p/rw-8-turning-podcasts-into-knowledge

2 3 4 Why Vector Databases Fail in LLM Projects: A Case for Graph-Based Retrieval | Dinis Cruz posted on the topic | LinkedIn
https://www.linkedin.com/posts/diniscruz_why-we-dont-use-vector-db-in-llm-projects-activity-7385592437508943872-MKys

5 Personalizing Audiobooks and Podcasts with graph-based models | Spotify Research
https://research.atspotify.com/2024/05/personalizing-audiobooks-and-podcasts-with-graph-based-models

6 7 8 Can LLMs Build Trustworthy Knowledge Graphs? Yes, With the Right Architecture
https://www.linkedin.com/pulse/can-llms-build-trustworthy-knowledge-graphs-yes-right-franco-iii-eedne?trk=public_post_comment-text